# Deckwatch Kenya API: what it is, how it works, and what you can build

Oct 8, 2026 · @Collins

## Overview

The Deckwatch Kenya API gives anyone free, keyless access to the public-safety data behind deckwatch.vercel.app: real incidents from 13 Kenyan newsrooms, county figures and monthly safety reports, as JSON.

It serves three kinds of data:

- **Incidents:** crimes, road crashes, floods, fires, protests, missing persons and other safety events, each placed in one of Kenya's 47 counties, rated for severity and scored for how well independent outlets corroborate it.
- **County figures:** a risk score and the last 24 hours for any county, and a trend over 7, 30 or 90 days compared with the period before.
- **Monthly safety reports:** each month's coverage consolidated into distinct incidents, with hotspots, a watch list and the sources behind every figure.

It is for developers building maps, alerts and dashboards; newsrooms and researchers who need a structured record of what was reported; and NGOs and civic-tech teams working on safety in Kenya. There is no sign-up, no key and no cost, and it can be called from any website.

## What you can build

The API is the raw material for tools that help people stay safe, report accurately and study what happens in Kenya.

| Who | What they can build | Endpoints they would use |
|---|---|---|
| App and web developers | A safety map or travel-check feature in a ride-hailing, logistics or travel app: "anything serious on this route today?" | List incidents, filtered by county, severity and `since` |
| Alerting services | SMS, WhatsApp or Telegram bots that post new critical incidents in a subscriber's county | List incidents, polled every few minutes |

| Who | What they can build | Endpoints they would use |
|---|---|---|
| Newsrooms | A live incident ticker on a website, or a monthly safety round-up built from the reports | List incidents, monthly reports |
| Researchers and students | Datasets of reported incidents by county, category and month, for studies of road safety, banditry or media coverage | Monthly reports, county trend |
| NGOs and county teams | Dashboards that track one county's incidents against the previous period | County at a glance, county trend |
| Civic-tech and hackathons | Prototypes, visualisations and teaching material, with no account or approval needed to start | All read endpoints |

Because every incident links to the original articles, anything built on the API can send readers back to the newsrooms that reported the story.

## Getting started

Send a GET request to `https://deckwatch.vercel.app`; there is nothing to register first.

- **Base URL:** `https://deckwatch.vercel.app`
- **Authentication:** none. Read endpoints are open to everyone.
- **From a browser:** any website may call the read endpoints (CORS allows every origin for GET).
- **Format:** JSON. An OpenAPI 3.1 description lives at `/api/openapi.json` for Postman, Swagger UI or a client generator.
- **Try it first:** the API page has a Try it button on every endpoint that sends a real request.

**curl**

```
curl "https://deckwatch.vercel.app/api/incidents?county=Nairobi&limit=5"
```

**JavaScript**

```
const res = await fetch("https://deckwatch.vercel.app/api/incidents?
severity=critical&limit=10");
const { results } = await res.json();

for (const incident of results) {
  console.log(incident.reportedAt, incident.county, incident.title);
}
```

**Python**

```
import requests

res = requests.get("https://deckwatch.vercel.app/api/incidents", params=
{"category": "flood", "limit": 20})
res.raise_for_status()

for incident in res.json()["results"]:
    print(incident["reportedAt"], incident["county"], incident["title"])
```

## Endpoint reference

Seven read endpoints answer GET requests from anywhere; one write endpoint accepts test submissions from Deckwatch itself.

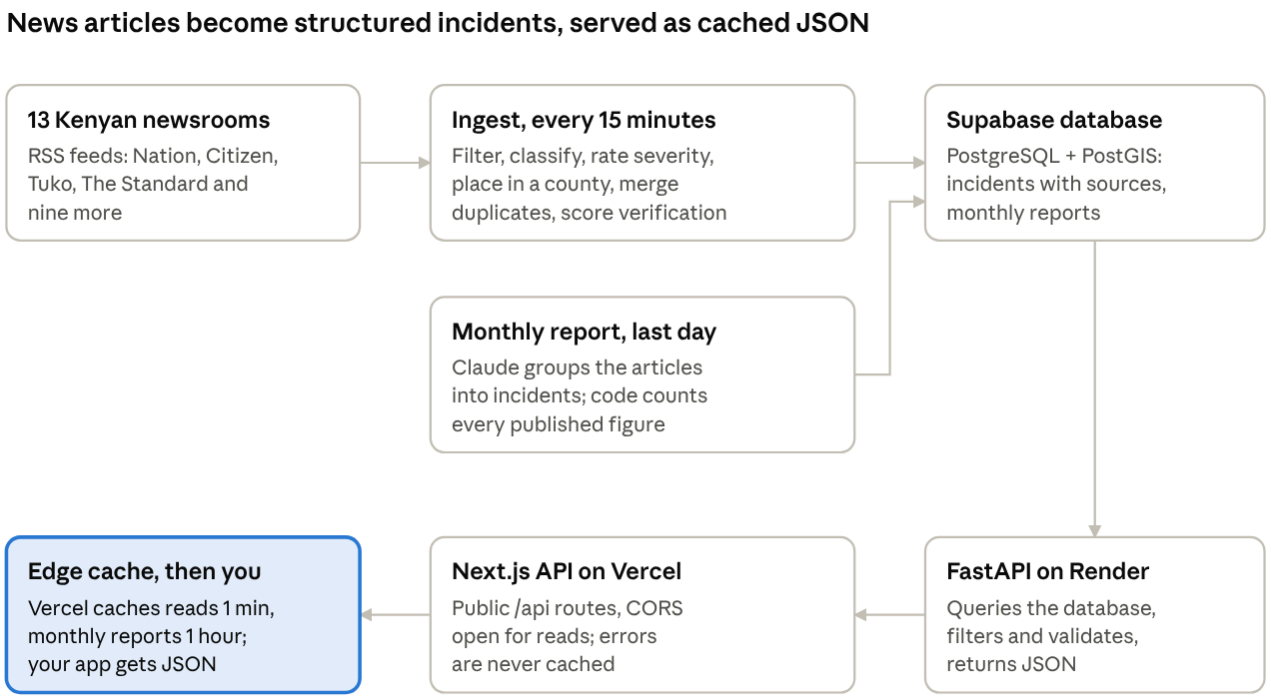
| Endpoint | What it returns | Parameters | Edge cache |
|---|---|---|---|
| `GET /api/incidents` | Real incidents, newest first, as summaries | `category`, `severity`, `county`, `verification`, `since` (ISO date), `limit` | 1 min |
| `GET /api/incidents/{id}` | One incident with its AI summary and recommended safety actions | `id` (e.g. `ing-cd5b3213`) | 1 min |
| `GET /api/counties/{slug}` | Risk score, incidents in the last 24 hours, top category | `slug` (e.g. `nairobi`, `homa-bay`) | 1 min |

| Endpoint | What it returns | Parameters | Edge cache |
|---|---|---|---|
| `GET /api/counties/{slug}/summary` | Counts for a period against the previous one, by category and severity, with a daily trend | `slug`, `days` = 7, 30 or 90 (default 30) | 1 min |
| `GET /api/monthly-reports` | Every published monthly report, as summaries | none | 10 min |
| `GET /api/monthly-reports/{month}` | One month's full report: incidents, sources, hotspots, watch list | `month` (`YYYY-MM`) | 1 h |
| `GET /api/openapi.json` | The OpenAPI 3.1 description of all of the above | none | 1 h |
| `POST /api/reports` | Validates a citizen report and returns an id; nothing is stored yet | JSON body: `category`, `description`, `lat`, `lng`, `anonymous` | not cached |

Filters on the incident list combine, and `limit` is applied after sorting, so `limit=10` returns the ten most recent matches. Errors come back as `{"error": "..."}` with status 400 for an invalid parameter and 404 for an unknown incident, county or month.

## How it works

A scheduled job turns Kenyan news articles into structured incidents in a database, and the API serves them through an edge cache, so most requests never reach the backend.




data flow · from newsrooms to your app

Articles flow right along the top into the database; requests flow back along the bottom to your app, with the edge cache answering repeat calls.

1. **Collect.** Every 15 minutes a GitHub Actions job reads the RSS feeds of 13 Kenyan newsrooms. Articles already on record are skipped, and a keyword pre-filter drops items that are plainly not about safety.
2. **Classify.** Each remaining article gets a category and a county, from keyword rules or, when configured, Claude with similar reviewed articles in view. Articles that aren't incidents, or name no Kenyan place, are dropped.
3. **Rate severity.** One written scale, from critical (three or more killed) to low (no one hurt, or coverage of a past event), applied by both classifiers.
4. **Merge duplicates.** A new article joins an existing incident when both are pinned to the same place within 8 km and 72 hours, or when their headlines tell the same story within 72 hours of the story's latest article. It is stored as an extra source rather than a new incident.
5. **Score verification.** Corroboration counts independent newsrooms: outlets with one owner count once, aggregators count for less.
6. **Report monthly.** On the last day of each month, Claude groups the month's articles into distinct incidents and writes the editorial notes, then code computes every figure from the articles themselves.

7. **Serve.** The Next.js routes on Vercel validate each request and proxy it to a FastAPI service on Render, which queries the Supabase database. Successful reads are cached at Vercel's edge, and every read endpoint allows calls from any website.

## Data model

Every incident has the same fields; the list endpoint omits only `aiSummary` and `recommendedActions`, which the single-incident endpoint adds.

| Field | Meaning |
|---|---|
| `id` | Stable id, `ing-` followed by a hash of the first article's link |
| `title` | Headline of the first article that reported it |
| `category` | One of 11 kinds (below) |
| `severity` | `low`, `medium`, `high` or `critical`, on the scale below |
| `county`, `locationName` | One of Kenya's 47 counties, and the most specific place named |
| `lat`, `lng` | Coordinates of that place; the county's centre when only the county is known |
| `reportedAt` | Publish time of the first article (ISO 8601, UTC) |
| `verificationScore`, `verificationStatus` | How well independent outlets corroborate it (0 to 100, and a status) |
| `sources` | Every article merged into the incident: outlet name, link and, for newer ones, headline and publish time |
| `reportCount` | Number of articles merged into it |
| `isLive` | Always `true` here: real reporting, never sample data |

**Categories:** crime, traffic_accident, flood, fire, unrest, missing_person, terror_alert, public_health, infrastructure, wildlife, election_violence.

**Severity** follows one written scale, used by both the keyword rules and the AI classifier:

| Severity | When |
|---|---|
| critical | Three or more people killed, a mass-casualty or terror attack, or a disaster forcing large evacuations |
| high | One or two people killed, or a shooting, stabbing or kidnapping |
| medium | People injured but no one killed, a missing person, or serious disruption |
| low | No one hurt (arrests, property damage, warnings), or coverage of past events such as court cases |

**Verification** counts independent newsrooms, not mastheads: outlets under one owner (for example Nation Africa and NTV Kenya) count once, and aggregators count for less. `verified` and `likely_true` mean several independent outlets reported it; `unconfirmed`, the most common status, usually means a single outlet.

**Monthly reports** are one JSON document per month. They carry the month's consolidated `events` (each with deaths, outlets and sources), `hotspots`, a `watch` list for the coming month, the articles set aside as context or off-topic, and how the coverage was counted (`articles`, `eventArticles`, `offTopic`, `coverageFrom`).

## Using the data responsibly

Deckwatch reports what Kenyan media publish, so anything built on it should show where a story came from and how sure the record is.

- **Credit the sources.** Credit "Deckwatch Kenya", and wherever you show an incident, link to the original articles in its `sources`.
- **Show the confidence.** Most incidents come from one outlet and are `unconfirmed`. Display `verificationStatus` next to an incident, and don't present a single-source report as established fact.
- **Expect approximate locations.** When an article names only a county, the incident sits at the county's centre. Don't draw it as an exact point on a street map.
- **Coverage follows newsrooms.** Most outlets are based in Nairobi, so remote counties are under-counted. More incidents in a county can mean more reporters, not more danger.

- **The live list counts reports more than events.** Duplicate merging catches only part of the repeated coverage of one story. For counts by month, the monthly reports are the consolidated, more reliable figures.
- **Cache on your side.** Responses are cached for up to a minute (an hour for monthly reports) and new articles arrive every 15 minutes at most, so polling more often than once a minute returns the same data.
- **Fair use.** There is no key and no published rate limit. Keep request volumes reasonable; heavy or bulk users should get in touch first. Abusive traffic may be blocked.
- **Not an emergency service.** In an emergency, contact the National Police Service on 999, 112 or 911.

## What's next

Version 1 is live; these are the improvements under consideration, none yet scheduled.

| Area | Change | Why it matters to developers |
|---|---|---|
| Access | Optional API keys with per-key usage limits | Reliable capacity for heavy users, and a way to reach them about changes |
| Abuse protection | A per-IP rate limit at the edge | Keeps the API fast for everyone |
| Citizen reports | Store and moderate submissions from `POST /api/reports` | First-hand reports alongside media coverage |
| Accuracy | Wider use of the AI classifier, merging of borderline duplicates, checks against fresh labelled samples | Fewer duplicate and mislabelled incidents in the live list |
| Freshness | A scheduler with a dependable 15-minute beat | Ingestion that keeps time instead of best-effort scheduling |

Responses may gain new fields, but existing fields won't change meaning without notice on the What's new page. Questions, bulk-access requests and bug reports go through the project's GitHub repository.